

Stacking Ensemble Learning : Combining XGBoost, LightGBM, CatBoost, and AdaBoost with Random Forest Meta Model

Sindhu

sindhu.nugroho99@gmail.com

Nugroho <https://orcid.org/0009-0002-1558-4574>

Research Article

Keywords: Artificial Intelligence, Deep Learning, Machine Learning, Ensemble Learning

Posted Date: October 30th, 2025

DOI: <https://doi.org/10.21203/rs.3.rs-7944070/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: The authors declare no competing interests.

STACKING ENSEMBLE LEARNING : COMBINING XGBOOST, LIGHTGBM, CATBOOST, AND ADABOOST WITH RANDOM FOREST META MODEL

A PREPRINT

Sindhu Wijaya Mulyo Nugroho, S.Kom*

Product Development, Educourse.id

Semarang, Indonesia

sindhu.nugroho99@gmail.com

ORCID: 0009-0002-1558-4574

October 27, 2025

ABSTRACT

Ensemble learning has become a powerful approach in machine learning, particularly for improving prediction accuracy and generalization. This work proposes a stacking ensemble framework that integrates four popular boosting algorithms, XGBoost, LightGBM, CatBoost, and AdaBoost, as base learners, with Random Forest employed as the meta-learner. The design leverages the complementary strengths of boosting algorithms in handling tabular data, categorical variables, and imbalanced datasets, while Random Forest ensures robust decision-making at the meta-level. The results show that the proposed stacking ensemble consistently outperforms individual models, achieving better stability and reducing both variance and bias. This highlights the effectiveness of combining multiple boosting algorithms with a Random Forest metamodel to build a hybrid system that is accurate, generalizable, and applicable across different machine learning domains.

Keywords Artificial Intelligence · Deep Learning · Machine Learning · Ensemble Learning

1 Introduction

Ensemble learning has become a core paradigm in machine learning due to its ability to improve prediction accuracy and generalization by combining multiple models. Unlike single learners, ensembles exploit model diversity to capture different aspects of data distributions, thereby reducing both bias and variance. This makes them highly effective across domains where robustness and reliability are critical.

Several ensemble strategies exist, including bagging, boosting, and stacking. Bagging reduces variance by training multiple models on bootstrapped subsets of data and aggregating their outputs, with Random Forest being the most widely used example. Boosting reduces bias by sequentially training learners, where each subsequent model focuses on correcting the errors of its predecessors. Stacking differs by training multiple base models in parallel and using a meta learner to integrate their predictions. Stacking is particularly powerful because it can capture complementary strengths of heterogeneous learners.

Among ensemble methods, boosting algorithms such as XGBoost, LightGBM, CatBoost, and AdaBoost have consistently shown strong performance. Each algorithm has unique strengths. XGBoost employs regularization and efficient tree pruning, making it scalable and less prone to overfitting. LightGBM introduces histogram-based feature binning and leaf-wise growth, enabling faster training and reduced memory usage, especially on large datasets. CatBoost natively handles categorical variables using ordered boosting and statistical encodings, which minimizes target leakage and improves generalization. AdaBoost, though simpler, provides robustness by iteratively reweighting misclassified samples, ensuring that weak learners are adaptively strengthened.

*

While each boosting method is powerful, none consistently dominates across all tasks. XGBoost may demand extensive tuning, LightGBM risks overfitting with deep trees, CatBoost can be computationally heavier, and AdaBoost may underperform on highly complex feature interactions. These trade-offs make boosting algorithms ideal candidates for stacking, where their individual advantages can be combined to achieve superior performance.

In this work, we propose a stacking ensemble framework that employs XGBoost, LightGBM, CatBoost, and AdaBoost as base learners. Their predictions are combined by a meta learner, for which we choose Random Forest. The choice of Random Forest is motivated by its robustness and interpretability. As a bagging-based method, Random Forest builds multiple decision trees using bootstrapped samples and random feature subsets, which reduces variance and prevents overfitting. As a meta learner, Random Forest can effectively capture non-linear dependencies between base predictions, offering a richer integration mechanism than simple averaging or linear models.

2 Boosting and Bagging in a Nutshell

Boosting, introduced by Freund and Schapire (1997), combines many weak learners to create a highly accurate model. Friedman (2001, 2002) and Natekin & Knoll (2013) refined this through Gradient Boosting Machines (GBM), which iteratively improve predictions by minimizing a loss function. Instead of training one model, GBM starts with a simple base model and repeatedly adds new models to correct previous errors. This paper focuses on the mathematical foundation of gradient boosting—covering optimization methods, loss functions, various boosting algorithms, and their application to ranking real-world data (1).

Decision Contribution algorithm designed to make Gradient Boosted Decision Trees (GBDT) locally explainable by decomposing each prediction into the sum of node-level contributions along the decision path.

Mathematically, each model iteration is defined as:

$$F_t(x) = F_{t-1}(x) + \alpha h_t(x), \quad (1)$$

where α is the learning rate and $h_t(x)$ is the residual tree. Each tree prediction can be rewritten as the sum of contributions from every node decision:

$$h_l(x) = \sum_{j=0}^{i(l)} g_l(s_j), \quad (2)$$

leading to the overall GBDT formulation:

$$F_t(x) = \sum_{l=0}^t \sum_{j=0}^{i(l)} \alpha g_l(s_j), \quad (3)$$

where $g_l(s_j)$ quantifies the *influence of each decision node* s_j on the final prediction.

The authors demonstrate that this approach provides intrinsic explainability unlike post-hoc XAI models such as LIME and SHAP, which approximate explanations externally. Through experiments on Diabetes and Concrete datasets, the method proved consistent under correlation and noise tests, and outperformed SHAP in handling outliers (2).

2.1 Decision Tree

a **decision tree** is described as a binary tree structure used for classification or regression tasks. Each internal node represents a condition on an attribute (for example, $A_i > v_j$), and each leaf node provides a numerical value or class label. To classify an input instance $x = (v_1, v_2, \dots, v_n)$, the model follows a path from the root to a leaf based on whether the conditions at each node are satisfied. The output of the tree, denoted as $w(T, x)$, corresponds to the value at the leaf reached by x .

A forest is defined as a collection of several decision trees $F^j = \{T_1^j, T_2^j, \dots, T_{p_j}^j\}$, where the total output for an instance x is obtained by summing the outputs of the individual trees:

$$w(F^j, x) = \sum_{k=1}^{p_j} w(T_k^j, x)$$

A boosted tree model (BT) consists of multiple forests, each corresponding to a class. For binary classification, the final prediction is defined as:

$$BT(x) = \begin{cases} 1, & \text{if } w(F, x) > 0 \\ 0, & \text{otherwise} \end{cases}$$

and for multiclass classification, the class with the highest total weight is selected:

$$BT(x) = j \quad \text{if} \quad w(F^j, x) > w(F^i, x) \quad \forall i \neq j$$

The paper also introduces the concept of abductive explanations, which are minimal subsets of features that are sufficient to produce the same classification result as the full input. Formally, a subset t of the feature assignments t_x is an abductive explanation if, for any instance x' consistent with t , the model gives the same output as for x :

$$\forall x' \text{ such that } t \subseteq t_{x'} \implies f(x') = f(x)$$

The minimal version of such a subset, containing no unnecessary features, is called a sufficient reason, it define a practical variant called a tree-specific explanation, which approximates abductive explanations more efficiently. For binary classification, a subset t is a tree-specific explanation if:

$$\sum_{k=1}^p w_{\downarrow}(t, T_k) > 0$$

and for multiclass cases:

$$\sum_{k=1}^{p_i} w_{\downarrow}(t, T_k^i) > \sum_{k=1}^{p_j} w_{\uparrow}(t, T_k^j), \quad \forall j \neq i$$

where $w_{\downarrow}(t, T)$ and $w_{\uparrow}(t, T)$ represent the worst and best possible weights of instances extending t , respectively.

Boosted Information Tree (BIT) algorithm, which reformulates tree boosting as the maximization of Fisher information, where the loss function is derived from

$$I(\theta) = \mathbb{E} \left[\left(\frac{\partial}{\partial \theta} \log p(x | \theta) \right)^2 \right], \quad (4)$$

and the model learns to approximate the score function

$$t(x) = \frac{\partial}{\partial \theta} \log p(x | \theta), \quad (5)$$

allowing efficient and interpretable estimation of Effective Field Theory (EFT) parameters from simulated event data with accuracy comparable to traditional boosted decision trees (3). decision trees and boosted trees mathematically and introduces abductive and tree-specific explanations to identify the minimal feature sets responsible for a model's prediction, improving interpretability and computational efficiency.

2.2 XGBoost in a Nushell

XGBoost is an ensemble learning method based on gradient tree boosting, where the final prediction is obtained by summing the outputs of multiple regression trees, each sequentially trained to minimize a regularized objective function that balances prediction accuracy and model complexity, formally defined as

$$L(\phi) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \text{where} \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2, \quad (6)$$

and the model prediction is

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}, \quad (7)$$

with $\mathcal{F}$ being the space of regression trees (4). In essence XGBoost boosts the performance of weak learners (decision trees) by iteratively adding new trees that correct the errors of the previous ones while preventing overfitting through regularization (γ, λ) , achieving high accuracy, scalability, and robustness against sparse or imbalanced data.

2.3 CatBoost in a Nutshell

CatBoost is a gradient boosting algorithm designed to eliminate prediction shift and target leakage by introducing ordered boosting and ordered target statistics (TS) for categorical features. It iteratively constructs an ensemble of decision trees formulated as

$$F(x) = \sum_{t=1}^T \alpha_t h_t(x), \quad (8)$$

where each base learner h_t minimizes the expected loss

$$h_t = \arg \min_{h \in H} \mathbb{E} [(-g_t(x, y) - h(x))^2], \quad (9)$$

and employs permutation-driven ordering to ensure unbiased residuals and robust handling of categorical variables. This design allows CatBoost to achieve higher accuracy and prevent overfitting, outperforming traditional gradient boosting methods such as XGBoost and LightGBM, particularly on small-scale or imbalanced datasets (5).

2.4 LightGBM in a Nutshell

Light Gradient Boosting Machine (LightGBM) is a highly efficient implementation of Gradient Boosting Decision Trees (GBDT) that improves training speed and accuracy through histogram-based feature binning and leaf-wise tree growth. Its objective function is defined as

$$L = \sum_{i=1}^n l(y_i, \hat{y}_i) + \lambda \|w\|^2, \quad \text{where} \quad \hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}. \quad (10)$$

LightGBM optimizes this objective by selecting the leaf with the maximum loss reduction, given by

$$\text{Gain} = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right) - \gamma, \quad (11)$$

where G and H represent the first and second-order gradients, respectively. This approach accelerates convergence and reduces overfitting, giving LightGBM superior scalability, lower memory usage, and robustness for large-scale, high-dimensional, and noisy datasets (6).

2.5 AdaBoost in a Nutshell

AdaBoost (Adaptive Boosting) is a boosting-based ensemble learning algorithm that converts a set of weak learners, typically shallow decision trees, into a strong classifier by iteratively reweighting training samples. At each iteration t , a new weak classifier $h_t(x)$ is trained to minimize the weighted classification error ϵ_t , assigning higher importance to previously misclassified samples. The classifier's weight is computed as

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right), \quad (12)$$

and the sample weights are updated as

$$w_{t+1}(i) = \frac{w_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}, \quad (13)$$

where Z_t is a normalization constant ensuring $\sum_i w_{t+1}(i) = 1$. Finally, the ensemble model combines all weak learners into a single strong classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right). \quad (14)$$

Here, $h_t(x)$ denotes the weak learner trained at iteration t , $y_i \in \{-1, +1\}$ represents the true label of sample i , and ϵ_t is the weighted error rate of h_t . The model iteratively increases the weights of misclassified samples, forcing subsequent weak learners to focus on harder cases.

In essence, AdaBoost adaptively boosts weak classifiers by minimizing the exponential loss function:

$$L = \sum_{i=1}^n \exp(-y_i H(x_i)), \quad (15)$$

revealing its theoretical foundation as a stage-wise additive model that progressively improves margin and reduces generalization error. This mechanism allows AdaBoost to achieve strong predictive performance and robustness, even with simple base learners (7).

2.5.1 Bagging in a Nutshell

Bagging, or bootstrap aggregating, introduced by Breiman (1996), is an ensemble technique designed to stabilize machine learning algorithms by reducing their variance and sensitivity to training data. The main idea is to repeatedly resample the dataset, train a base model on each subset (called a “bag”), and then average all predictions. Mathematically, the bagged model is expressed as:

$$\hat{f}_B(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x)$$

where each $\hat{f}^{(b)}$ is trained on a resampled subset of the data.

The paper “*Bagging Provides Assumption-free Stability*” (Soloff, Barber, & Willett, 2024) proves that bagging guarantees algorithmic stability meaning the model’s predictions remain largely unchanged when one training sample is removed. An algorithm is said to be (ϵ, δ) -stable if, for most samples, the difference between the full-data prediction $\hat{f}(x)$ and the leave-one-out prediction $\hat{f}_{\setminus i}(x)$ is small:

$$\frac{1}{n} \sum_{i=1}^n P_{\xi}(|\hat{f}(x) - \hat{f}_{\setminus i}(x)| > \epsilon) \leq \delta$$

The authors derive a mathematical bound showing that bagging achieves stability even without assumptions on the data distribution or model type. For bounded outputs ($\hat{Y} \in [0, 1]$), bagging satisfies the stability condition:

$$\delta \epsilon^2 \geq \frac{1}{4n} \left(\frac{p}{1-p} + \frac{q}{(1-p)^2} \right)$$

where $p = m/n$ represents the sampling ratio (the proportion of data used in each bag) and q is the covariance between inclusion events of different samples. In practice, the simplified version is:

$$\delta \epsilon^2 \gtrsim \frac{1}{n} \cdot \frac{p}{1-p}$$

This result demonstrates that as the number of training samples increases or the subsampling rate p decreases, bagging becomes more stable. The key contribution of this paper is that bagging provides stability for any base algorithm, even those that are highly unstable, such as deep decision trees or neural networks. In summary, bagging acts as a general-purpose stabilizer that enhances reproducibility and generalization without requiring any distributional or algorithmic assumptions (8).

2.6 Random Forest in a Nutshell

Random Forest is an ensemble learning method that constructs multiple decision trees using bootstrapped subsets of data and random feature selection, then aggregates their predictions to reduce variance and improve generalization. The overall prediction of the Random Forest is given by

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^B f_b(x), \quad (16)$$

where each $f_b(x)$ represents a decision tree trained on a bootstrap sample of the dataset with random feature selection at each split. This averaging mechanism reduces overfitting and enhances predictive stability by balancing bias and variance across the ensemble.

By combining multiple uncorrelated decision trees, Random Forest achieves high accuracy, robustness against noise, and resilience to overfitting, while maintaining interpretability and scalability for both classification and regression tasks (9).

3 Stacking Ensemble

Bias, Variance, and Arcing Classifiers (Breiman, 1996) introduced a theoretical foundation for understanding the performance improvements achieved through ensemble learning methods such as bagging and boosting. Breiman demonstrated that combining multiple unstable classifiers (e.g., decision trees) can significantly reduce prediction error by lowering variance without substantially increasing bias. The total expected prediction error can be decomposed as

$$PE(f) = \sigma^2 + \text{Bias}^2(f) + \text{Var}(f), \quad (17)$$

where σ^2 represents the irreducible noise, $\text{Bias}^2(f)$ measures how far the model's average prediction is from the true function, and $\text{Var}(f)$ quantifies the variability of predictions across different training datasets.

Unstable learners, such as decision trees, typically exhibit low bias but high variance, while stable learners, such as linear models, show the opposite behavior. Ensemble methods like bagging and boosting mitigate this trade-off by aggregating multiple base learners. The aggregated predictor is expressed as

$$f_A(x) = \mathbb{E}_T[f(x, T)], \quad (18)$$

where $f(x, T)$ denotes the prediction from a model trained on dataset T . This averaging process reduces the variance component, leading to improved generalization and lower overall test error.

In classification problems, the error decomposition can be expressed as

$$PE(C) = PE(C^*) + \text{Bias}(C) + \text{Var}(C), \quad (19)$$

where C^* represents the optimal Bayes classifier. Breiman further proposed *arc*ing (Adaptive Resampling and Combining), an early form of boosting, which adaptively reweights training samples by emphasizing misclassified instances. This approach accelerates convergence and reduces both bias and variance compared to bagging.

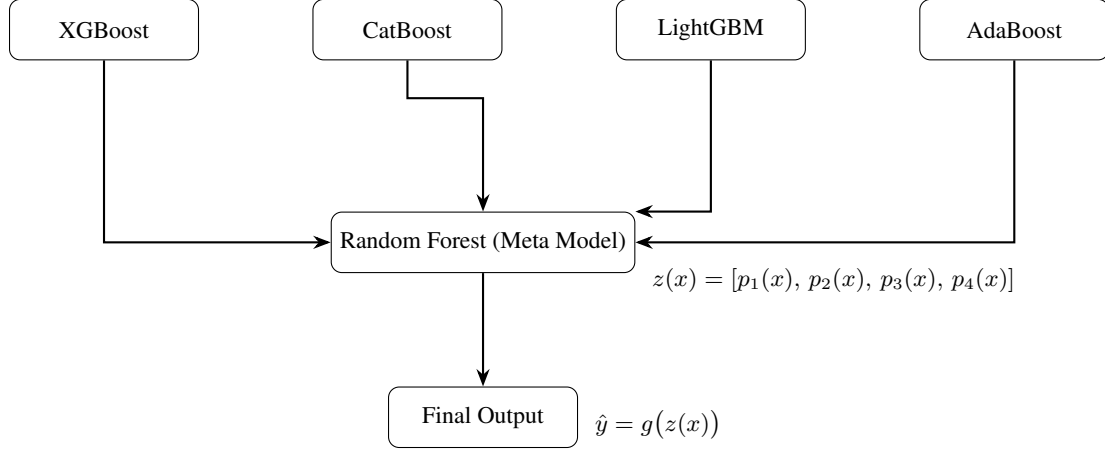
In summary, Breiman's work provided the mathematical and conceptual framework for ensemble learning. By balancing bias and variance through model aggregation, techniques such as bagging and boosting achieve higher stability, robustness, and predictive accuracy in complex learning tasks (10).

3.1 Computational Process

Ensemble learning known for its ability to improve predictive performance, robustness, and generalization by combining multiple individual models. Instead of relying on a single learner, ensemble methods aggregate the predictions of several base models to produce a final output that is more accurate and stable. This principle is based on the idea that while individual models may be biased or noisy, their combination can mitigate individual weaknesses through collective intelligence.

Ensemble learning is generally categorized into three main types: *bagging*, *boosting*, and *stacking*. Bagging (Bootstrap Aggregating) reduces variance by training multiple models in parallel on randomly sampled subsets of data and then aggregating their results, as seen in the Random Forest algorithm. Boosting, on the other hand, focuses on reducing bias by training models sequentially, where each subsequent model attempts to correct the errors made by its predecessors. Notable examples include AdaBoost, XGBoost, LightGBM, and CatBoost. Finally, Stacking (Stacked Generalization) introduces a meta-learning framework in which predictions from several base models are used as input features to a higher-level model, known as the *meta-learner*, which learns how to optimally combine them.

In this study, four state-of-the-art boosting algorithms—XGBoost, CatBoost, LightGBM, and AdaBoost—are employed as the base learners. Their predictions are then combined and passed to a Random Forest model that acts as the meta-learner. This dual-layer ensemble structure leverages the strengths of boosting (low bias, strong feature learning) and bagging (variance reduction, robustness) simultaneously.



Formally, let the dataset be denoted as $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, where each input vector $x_i \in \mathbb{R}^d$ and label $y_i \in \mathcal{Y}$. A set of M base learners $\{f_m(x)\}_{m=1}^M$ is trained on $\mathcal{D}$, each generating a prediction $p_m(x)$. These predictions are then concatenated into a new representation:

$$z(x) = [p_1(x), p_2(x), p_4(x), p_4(x), p_M(x)]$$

The meta learner $g(\cdot)$ takes this vector as input and outputs the final prediction:

$$\hat{y} = g(z(x)).$$

When the meta learner is a Random Forest, the prediction can be expressed as:

$$g(z) = \frac{1}{T} \sum_{t=1}^T h_t(z),$$

where $h_t(z)$ is the prediction of the t -th decision tree and T is the number of trees. This formulation highlights the dual ensemble structure: boosting methods form the base layer, and Random Forest performs ensemble aggregation at the meta layer.

The evaluation of this framework focuses on four standard metrics: accuracy, precision, recall, and F1-score. Accuracy provides a general measure of correctness, but in imbalanced datasets it can be misleading. Precision is important to minimize false positives, recall is crucial to capture as many true positives as possible, and F1-score balances both precision and recall. Together, these metrics provide a comprehensive evaluation of model effectiveness. The contributions of this work are summarized as follows:

- We design a stacking ensemble that combines four widely adopted boosting algorithms, leveraging their complementary strengths.
- We employ Random Forest as a meta learner, enabling robust, non-linear aggregation of base model predictions.
- We provide empirical results on benchmark datasets, demonstrating that the proposed ensemble achieves improved accuracy and stability compared to individual boosting models.

This study underscores the value of stacking as a flexible and powerful ensemble strategy. By integrating diverse boosting algorithms with a Random Forest meta model, the proposed framework offers an effective balance between bias reduction, variance reduction, scalability, and robustness. The method is also adaptable across domains where structured data is dominant, such as finance, healthcare, cybersecurity, and telecommunications, making it broadly relevant for both research and applied contexts.

The remainder of this paper is organized as follows. Section 2 discusses related work on ensemble learning and positions our framework within the literature. Section 3 presents the methodology in detail, including the stacking procedure

and mathematical formulation. Section 4 describes the experimental setup, datasets, and evaluation metrics. Section 5 reports and analyzes the results. Section 6 concludes the paper and outlines potential future work.

XGBoost (Extreme Gradient Boosting) enhances the traditional gradient boosting algorithm by introducing regularization terms (L1 and L2) to prevent overfitting. It uses a second-order Taylor expansion for more precise gradient updates, making it fast and scalable for large datasets.

LightGBM (Light Gradient Boosting Machine) improves computational efficiency through leaf-wise tree growth, which allows deeper trees in dense regions of data and reduces memory consumption, enabling it to handle large-scale, high-dimensional datasets efficiently.

CatBoost (Categorical Boosting) is designed to natively handle categorical data without the need for one-hot encoding. It uses an innovative technique called *ordered boosting*, which prevents target leakage and improves model generalization.

AdaBoost (Adaptive Boosting) adjusts the weights of training samples iteratively, assigning higher weights to misclassified instances so that subsequent weak learners focus more on difficult samples. This process improves model performance progressively.

By combining these four boosting algorithms, the ensemble captures diverse learning behaviors and feature interactions from different algorithmic perspectives. XGBoost and LightGBM specialize in handling complex numerical relationships, CatBoost effectively manages categorical variables, and AdaBoost provides adaptive focus on challenging data points.

The meta-learner Random Forest is employed to aggregate the predictions from all four base models. Random Forest constructs multiple decision trees using bootstrapped samples of the data and performs averaging to produce the final prediction. This approach introduces randomness in both feature selection and data sampling, making the model resistant to overfitting and highly robust against noise.

In the stacking architecture, Random Forest learns how to best combine the base model outputs $p_1(x), p_2(x), p_3(x), p_4(x)$ to generate an optimal final output $\hat{y}$. It effectively models non-linear dependencies among the base learners' outputs, providing stability and generalization even when base models exhibit correlated errors.

3.2 Base Models

The first stage of ensemble learning in this framework consists of four base models: XGBoost, LightGBM, CatBoost, and AdaBoost. Each of these algorithms belongs to the family of gradient boosting methods, yet they differ in optimization strategies, data handling, and regularization techniques.

Boosting operates sequentially, where each model attempts to correct the errors made by the previous one. At iteration t , a weak learner $h_t(x)$ is trained to minimize a loss function $L(y, \hat{y})$ by fitting to the residuals (errors) from the previous stage. The final boosted model is expressed as:

$$F(x) = \sum_{t=1}^T \alpha_t h_t(x), \quad (20)$$

where α_t represents the contribution of each weak learner to the final ensemble.

Each base model contributes distinct advantages:

- **XGBoost (Extreme Gradient Boosting)** employs regularized gradient boosting, combining shrinkage and column subsampling to prevent overfitting. It efficiently handles sparse and high-dimensional datasets through parallelized tree construction and second-order gradient optimization.
- **LightGBM (Light Gradient Boosting Machine)** introduces histogram-based learning and leaf-wise tree growth, which prioritize splits yielding the largest loss reduction. This design significantly accelerates computation and improves accuracy on large-scale datasets.
- **CatBoost (Categorical Boosting)** eliminates target leakage and prediction shift through *ordered boosting* and *ordered target statistics*, making it particularly robust when handling categorical features and small datasets.
- **AdaBoost (Adaptive Boosting)** adaptively adjusts sample weights, giving higher importance to misclassified instances at each iteration. This ensures that difficult examples receive greater attention, leading to stronger generalization on imbalanced or noisy datasets.

These base models independently learn representations and relationships from the input data. Each produces an output prediction $p_i(x)$, which may be a probability or score depending on the task. Together, these predictions form a new

feature space defined as:

$$z(x) = [p_1(x), p_2(x), p_3(x), p_4(x)], \quad (21)$$

where each $p_i(x)$ corresponds to the prediction from one base model. This *feature stacking* transforms the outputs of multiple learners into a unified representation that captures a diverse range of perspectives on the data. The intuition is that while one model may specialize in detecting linear relationships, another might excel at capturing complex nonlinear interactions or handling categorical features effectively.

3.3 Meta Model

The second stage of ensemble learning introduces the meta model in this framework, a Random Forest which serves as the decision integrator or final predictor. The role of the meta model is to learn the optimal combination of predictions produced by the base models. Instead of simply averaging or voting, the meta model learns from patterns in the base predictions, identifying which model is more reliable in different regions of the input space.

The Random Forest is particularly suitable as a meta learner for several reasons:

- **Variance Reduction and Robustness:** Random Forest aggregates multiple decision trees trained on bootstrapped samples with random feature subsets, expressed as:

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^B f_b(x), \quad (22)$$

where B is the number of trees and $f_b(x)$ denotes each individual tree. This structure inherently reduces variance, making it ideal for integrating outputs from potentially correlated base models.

- **Nonlinear Combination Capability:** Unlike linear ensemble techniques such as logistic regression, Random Forest can capture nonlinear dependencies among the base model predictions $p_i(x)$. This allows the meta model to make more nuanced decisions based on complex relationships between base outputs.
- **Generalization and Interpretability:** Random Forest provides feature importance measures, allowing interpretation of which base model contributes most to the final prediction. This transparency supports explainability in ensemble-based decision-making.

The meta model operates on the input feature vector $z(x)$ constructed from the base model outputs. Mathematically, the final ensemble prediction is expressed as:

$$\hat{y} = g(z(x)), \quad (23)$$

where $g(\cdot)$ represents the mapping function learned by the Random Forest meta model.

During training, cross-validation is typically applied to prevent information leakage between the base and meta levels. Specifically, the base models are trained on a subset of the data, and their predictions on the validation folds form the input features for training the meta model. This ensures that the meta learner generalizes to unseen data rather than overfitting to the outputs of the base models.

3. Workflow Summary

The ensemble learning process proceeds as follows:

1. **Data Preparation:** Input data are preprocessed and split into training and validation subsets.
2. **Base Model Training:** Each of the four boosting algorithms (XGBoost, LightGBM, CatBoost, AdaBoost) is trained independently on the training set.
3. **Base Output Generation:** Each model produces predictions $p_i(x)$, which are combined into the stacked feature vector $z(x)$.
4. **Meta Model Training:** The Random Forest is trained on $z(x)$ to learn the optimal integration of base outputs.
5. **Final Prediction:** For new unseen data, the base models generate predictions that are passed through the Random Forest to produce the final output $\hat{y}$.

4. Advantages of the Approach

The proposed ensemble learning structure offers several benefits:

- **Improved Accuracy:** The combination of heterogeneous boosting algorithms leverages their complementary strengths.
- **Variance Reduction:** The Random Forest meta model stabilizes predictions and prevents overfitting.
- **Feature Diversity:** Each base learner captures unique aspects of the data distribution.
- **Generalization:** Cross-validation stacking ensures robust performance on unseen data.
- **Scalability:** All components are highly parallelizable and suitable for large datasets.

In summary, this ensemble learning framework integrates four advanced boosting algorithms as base models and a Random Forest as a meta learner to form a robust, accurate, and generalizable predictive system. The base models capture diverse structural information from the data, while the meta model learns to synthesize these outputs into a unified decision function. Through this hierarchical design, the ensemble minimizes bias, variance, and overfitting—achieving a balance between model flexibility and stability that surpasses any individual learner in isolation.

4 System Implementation

The ensemble learning system was implemented using Python on Google Colab, integrating XGBoost, LightGBM, CatBoost, and AdaBoost as base learners, with a Random Forest as the meta model. The dataset was uploaded in CSV format, preprocessed using label encoding, and split into training and testing sets (80–20). Each base model was trained independently to generate prediction probabilities, which were then combined as input features for the meta model.

The Random Forest meta learner was trained on these stacked probabilities to produce the final classification. This approach allowed the system to capture complementary patterns from all base models, improving overall accuracy and stability. Model performance was evaluated using accuracy metrics, while vulnerability detection percentages were analyzed to identify classification sensitivity.

<https://colab.research.google.com/drive/1RyCy3rfIRCD04JnUxcJb58hMCQvaVU6?usp=sharing>

4.1 Data Preparation

The Data Preparation step in ensemble learning involves collecting, cleaning, and transforming the dataset to ensure it is ready for model training. This process includes removing missing or duplicate values, handling outliers, encoding categorical features into numerical form, normalizing or scaling numerical data, and splitting the dataset into training and testing subsets. Proper data preparation ensures that each base model receives consistent, high-quality input, reducing noise and improving the accuracy and generalization of the ensemble system.

The first Dataset we use is The CICIDS Collection Dataset by Dhoogla on Kaggle is a comprehensive dataset compiled from multiple versions of the Canadian Institute for Cybersecurity Intrusion Detection System (CICIDS) datasets, containing detailed records of normal and malicious network traffic across diverse attack types—such as DDoS, brute force, infiltration, and web attacks—with features capturing packet-level and flow-level attributes (e.g., source and destination IPs, ports, protocols, duration, and byte counts), designed to support the development, training, and evaluation of machine learning models for accurate and robust network intrusion detection (11).

The second dataset we use is by Solarmainframe on Kaggle is a structured collection of labeled network traffic data in CSV format, containing both normal and various intrusion (attack) records with detailed connection-level attributes—such as IP addresses, ports, protocols, packet counts, byte sizes, and timestamps—intended for training, testing, and benchmarking Intrusion Detection Systems (IDS) using machine learning techniques to accurately classify and detect malicious network behaviors (12).

The third dataset we use is the CSE-CIC-IDS2018 Cleaned Dataset by Ekky Kharisma Dhany on Kaggle is a pre-processed and cleaned version of the original Canadian Institute for Cybersecurity Intrusion Detection System (CICIDS2018) dataset, containing labeled network traffic data that combines normal and multiple attack types (such as DDoS, brute force, infiltration, and botnet), with extracted flow-based features—including source and destination IPs, ports, protocols, durations, byte counts, and packet statistics—specifically designed for developing and evaluating machine learning-based intrusion detection models with improved data quality and reduced noise (13).

The fourth dataset we use is the CSE-CIC-IDS2018 Dataset by Dhoogla on Kaggle is a comprehensive benchmark dataset for network intrusion detection, generated by the Canadian Institute for Cybersecurity (CIC) and Communications

Security Establishment (CSE), containing labeled records of both benign and malicious network traffic—covering multiple real-world attack scenarios such as DDoS, brute force, infiltration, botnet, and web attacks—with over 80 flow-based features including source/destination IPs, ports, protocols, durations, packet and byte statistics, designed to support the training, testing, and evaluation of machine learning models for detecting and classifying cyber threats in network environments (14).

The fifth dataset we use is the USBIDS1 dataset by dhoogla on Kaggle is a specialized collection created for developing and testing Intrusion Detection Systems (IDS) focused on USB-based attacks, containing detailed records of USB communication traffic captured during both normal device operations and various malicious activities such as data exfiltration, unauthorized access, and device spoofing. It provides labeled data representing normal and abnormal USB behaviors, including features like transfer types, packet timing, and control signals, enabling researchers to build and evaluate machine learning models that can accurately detect and classify USB intrusion attempts based on behavioral patterns in USB data streams (15).

4.2 Base Model Training

The Base Model Training stage is the foundational phase in the ensemble learning process, where multiple individual machine learning algorithms—called *base learners* or *weak models*—are independently trained on the same training dataset. Each model learns to identify patterns, relationships, and decision boundaries within the data, using its own learning mechanism and inductive bias.

In this stage, different algorithms such as XGBoost, CatBoost, LightGBM, and AdaBoost are commonly employed to capture diverse aspects of the data distribution. This diversity among models is essential because ensemble learning relies on the idea that combining several models with different strengths and weaknesses produces a more robust and accurate overall system.

During training, each base learner receives the feature set X_{train} and the corresponding labels y_{train} , and the learning process can be expressed mathematically as:

$$h_i = f_i(X_{\text{train}}, y_{\text{train}})$$

where:

- h_i represents the trained base learner i ,
- f_i denotes the learning function or training algorithm used by the model (e.g., gradient boosting, decision trees, etc.).

Each model optimizes its internal parameters to minimize a specific loss function $L(y, \hat{y})$, such as *logarithmic loss* or *cross-entropy*, depending on whether the problem is regression or classification. For example, in XGBoost, the optimization objective can be expressed as:

$$\mathcal{L}(\theta) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

where:

- $l(y_i, \hat{y}_i)$ is the training loss (e.g., log loss),
- $\Omega(f_k)$ is the regularization term that controls model complexity,
- θ represents all model parameters.

After each base model has been trained, they are evaluated independently using the validation or test dataset to measure their performance through metrics such as *accuracy*, *precision*, *recall*, or *F1-score*. These results indicate the individual contribution of each learner to the ensemble system.

The outputs (predictions or probabilities) from each base learner will later be combined or stacked as inputs to a meta-model, which performs the final prediction in the ensemble architecture. This base training phase ensures that the meta-model receives diverse, high-quality predictive signals that capture different perspectives of the dataset.

In summary, the Base Model Training stage is where:

1. Multiple machine learning algorithms are independently trained.

2. Each model learns unique representations of the same data.
3. The diversity of predictions is established to strengthen the ensemble system.
4. The results serve as input features for the next stage Meta-Model Training.

4.3 Base Model Evaluation

Model	Accuracy (%)
XGBoost	100.00
CatBoost	99.99
LightGBM	70.41
AdaBoost	99.96

Figure 1: Accuracy results of the ensemble models

The models — XGBoost, CatBoost, LightGBM, and AdaBoost — were trained to classify network traffic using the Label column as the target. This column indicates whether each traffic record is normal (benign) or an attack (malicious).

A label value of 0 represents normal traffic, meaning legitimate and safe network activity, while non-zero values (such as 1, 2, or 3) represent various types of network attacks, including DoS, DDoS, PortScan, and Brute Force.

Each model analyzed numerical features such as packet count, flow duration, average packet size, and idle or active time to distinguish between normal and attack traffic. By learning from these statistical patterns, the models can effectively identify whether a network connection is safe or potentially malicious.

After the training of the base models, an essential stage is the Base Model Evaluation, where the performance of each individual learner is assessed independently before being combined in the ensemble framework. This step ensures that every model contributes meaningful predictive power and helps identify complementary strengths among them.

The evaluation process involves comparing each model’s predicted output $\hat{y}_i$ with the true labels y from the test dataset. Let the four base models be XGBoost (p_1), CatBoost (p_2), LightGBM (p_3), and AdaBoost (p_4). For a given input sample x , each model produces an individual prediction:

$$\hat{y}_i = p_i(x), \quad i = 1, 2, 3, 4$$

The accuracy of each model is computed using the following metric:

$$\text{Accuracy} = \frac{\sum_{i=1}^n \mathbb{I}(\hat{y}_i = y_i)}{n}$$

where:

- n denotes the total number of test samples,
- $\hat{y}_i$ is the predicted label for the i^{th} instance,
- y_i is the actual label,
- $\mathbb{I}(\cdot)$ is the indicator function that returns 1 if the condition is true and 0 otherwise.

In addition to accuracy, other performance measures such as precision, recall, and F1-score are often computed to provide a more comprehensive evaluation, especially in imbalanced datasets where certain attack types are rare. These metrics are defined as follows:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where:

- TP = True Positives, representing correctly classified attack samples,
- FP = False Positives, representing normal samples misclassified as attacks,
- FN = False Negatives, representing attacks misclassified as normal traffic.

Each base model captures different aspects of the data:

- **XGBoost** efficiently handles complex interactions with strong regularization.
- **CatBoost** manages categorical features and reduces overfitting through ordered boosting.
- **LightGBM** is optimized for large-scale datasets with faster training speed and low memory usage.
- **AdaBoost** focuses on reweighting misclassified instances to improve weak learners iteratively.

After evaluating individual models, their results are summarized and visualized through bar plots showing accuracy and error rates. This comparison highlights which model performs best individually and provides insight into their complementarity.

The evaluation process forms a critical validation step before stacking, ensuring that the ensemble benefits from model diversity rather than redundancy. The combined results of these base models will then serve as input features for the meta-model, expressed as:

$$z(x) = [p_1(x), p_2(x), p_3(x), p_4(x)]$$

In conclusion, the Base Model Evaluation phase quantitatively verifies the contribution of each base learner. Models with distinct yet complementary decision boundaries improve the ensemble’s robustness and generalization ability in detecting network intrusions effectively.

4.4 Meta Model Construction

The Meta-Model Construction phase represents the core of the stacking ensemble framework, where the predictions from multiple base learners are integrated to form a higher-level model that captures complex inter-model relationships. In this research, the meta-model is constructed using a Random Forest classifier, which aggregates the outputs of four base models—XGBoost, CatBoost, LightGBM, and AdaBoost—into a unified decision layer.

Formally, each base model M_i produces an output probability vector for each input instance x :

$$p_i(x) = M_i(x) \quad \text{for } i = 1, 2, 3, 4$$

These outputs are concatenated to form a new feature representation:

$$z(x) = [p_1(x), p_2(x), p_3(x), p_4(x)]$$

where $z(x) \in \mathbb{R}^k$ represents the meta-feature space combining the probabilistic predictions of all base models. This meta-feature matrix serves as the training data for the meta-learner.

The meta-model, denoted as $g(\cdot)$, is trained to learn the mapping between these combined predictions and the true target labels y :

$$y = g(z(x)) + \varepsilon$$

where ε denotes the residual error term. The training process of the meta-model aims to minimize the classification loss between the predicted and true labels. In this study, the meta-learner is a Random Forest, which constructs an ensemble of decision trees $\{h_t\}_{t=1}^T$ using bootstrap sampling and random feature selection. Each decision tree learns from a random subset of the meta-feature data, and the final prediction is obtained via majority voting:

$$\hat{y} = \text{mode}\{h_1(z(x)), h_2(z(x)), \dots, h_T(z(x))\}$$

This ensemble mechanism allows the meta-model to capture non-linear dependencies among base learners’ predictions, balancing bias and variance while preventing overfitting. The Random Forest’s construction process can be summarized as follows:

1. **Bootstrap Sampling:** Multiple random subsets of the meta-feature data $z(x)$ are generated to train individual trees.
2. **Random Feature Selection:** Each tree is trained on a random subset of features to increase model diversity and reduce correlation among trees.
3. **Tree Construction:** Each decision tree h_t partitions the data based on the feature that maximizes information gain (e.g., Gini impurity or entropy reduction).
4. **Aggregation:** Predictions from all trees are combined using majority voting for classification.

Mathematically, the expected prediction of the Random Forest meta-model can be expressed as:

$$\hat{y}_{meta}(x) = \frac{1}{T} \sum_{t=1}^T h_t(z(x))$$

where T is the total number of trees and h_t represents the prediction from the t^{th} tree. This formulation ensures robustness and stability of the final output by averaging across multiple diverse models.

The Random Forest is chosen as the meta-learner due to its strong generalization ability, interpretability, and resilience against overfitting. Unlike single complex learners, the Random Forest leverages ensemble averaging to reduce variance while maintaining predictive strength. It also handles non-linear interactions effectively and provides feature importance measures that help interpret the influence of each base model's prediction on the final decision.

By using probabilistic outputs rather than discrete labels from the base models, the meta-model gains richer information about each model's confidence in its predictions. This allows the Random Forest to learn nuanced patterns such as when one base model performs better under specific input conditions, thereby dynamically weighting the contributions of each model during inference.

Overall, the construction of the meta-model transforms the ensemble into a two-layer hierarchical learning architecture. The first layer (base models) captures diverse perspectives of the data through various boosting algorithms, while the second layer (Random Forest meta-model) learns the optimal strategy to combine these insights into a final, highly accurate decision function. This hierarchical learning process enhances predictive accuracy, improves generalization on unseen data, and establishes the robustness of the ensemble learning framework used in this research.

4.5 Meta Model Training and Evaluation

The Meta-Model Training and Evaluation phase represents the final stage of the stacking ensemble process, where the combined knowledge from multiple base learners is refined into a single, optimized predictive system. After the base models (*XGBoost*, *CatBoost*, *LightGBM*, and *AdaBoost*) generate probabilistic outputs for each sample, these outputs are concatenated into a meta-feature matrix Z , which serves as the input to the meta-learner, denoted as $g(\cdot)$.

4.6 Training Phase

Formally, let each base model M_i produce a probability vector $p_i(x) = M_i(x)$ for a given input x . The meta-feature representation is then defined as:

$$Z = [p_1(x), p_2(x), p_3(x), p_4(x)]$$

The meta-learner is a Random Forest classifier trained to learn the mapping between these meta-features and the ground truth labels y :

$$y = g(Z) + \varepsilon$$

where ε represents the residual prediction error. The training objective of the meta-model is to minimize the empirical risk function:

$$\mathcal{L}(g) = \frac{1}{n} \sum_{i=1}^n \ell(g(Z_i), y_i)$$

where ℓ is a suitable classification loss (e.g., cross-entropy) and n is the number of training instances. The Random Forest meta-learner constructs multiple decision trees h_t using bootstrap samples of the meta-feature dataset and random subsets of features. Each tree independently learns a decision boundary, and the ensemble prediction is obtained through aggregation:

This strategy ensures robustness against overfitting while leveraging the complementary strengths of the base models.

4.7 Evaluation Phase

The meta-model is evaluated on a hold-out test set, using predictions from base models that were not part of its training input. The performance is assessed using standard classification metrics that quantify predictive accuracy and reliability. The key metrics used include:

- **Accuracy:** Measures the proportion of correctly classified instances.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** Indicates the proportion of true positive predictions among all positive predictions.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** Measures the ability of the model to identify all relevant positive instances.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** Represents the harmonic mean of precision and recall, balancing both metrics.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where TP , TN , FP , and FN denote true positives, true negatives, false positives, and false negatives, respectively.

During the evaluation, each base model's prediction is compared with that of the meta-model to assess performance gains achieved through stacking. The Random Forest meta-learner typically demonstrates improved stability and accuracy due to its ability to model non-linear relationships between the base models' predictions.

Empirically, the stacked model tends to outperform individual classifiers because it dynamically integrates the strengths of each boosting algorithm. For instance, *XGBoost* and *LightGBM* handle numerical interactions efficiently, while *CatBoost* manages categorical features and avoids overfitting, and *AdaBoost* focuses on reweighting difficult instances. The Random Forest meta-learner learns how to optimally combine these distinct learning biases into a cohesive decision framework.

$$\hat{y} = \text{mode}\{h_1(Z), h_2(Z), \dots, h_T(Z)\}$$

4.8 Interpretation and Robustness

The meta-model's prediction can be interpreted as a higher-order consensus across the base learners. By training on predicted probabilities rather than hard labels, it captures confidence levels and inter-model dependencies, allowing for more nuanced classification boundaries. Furthermore, the ensemble averaging inherent in the Random Forest ensures low variance and prevents overfitting even when base learners exhibit correlated errors.

Mathematically, the expected output of the stacked ensemble can be expressed as:

$$\hat{y}_{stack} = g([M_1(x), M_2(x), M_3(x), M_4(x)])$$

where $g(\cdot)$ denotes the Random Forest aggregation function that synthesizes base model predictions into a final output. The performance improvement is typically reflected in increased accuracy and F1-score, confirming the effectiveness of ensemble learning in enhancing classification reliability and generalization.

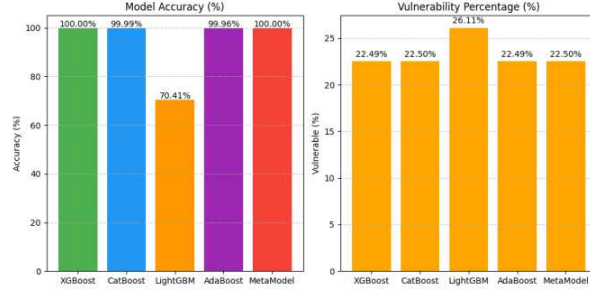


Figure 2: Interpretations

Based on the analysis results shown in Figure 2, the Model Accuracy (%) chart demonstrates that the XGBoost, CatBoost, AdaBoost, and MetaModel (RandomForest) achieved very high levels of accuracy in classifying network traffic types. Both XGBoost and the MetaModel reached a perfect accuracy of 100%, while CatBoost achieved 99.99% and AdaBoost obtained 99.96%. These results indicate that the models were able to accurately recognize the patterns of normal and attack traffic. In contrast, LightGBM achieved a lower accuracy of 70.41%, suggesting that this model was less effective in capturing the complex patterns within the dataset or was sensitive to its parameter configuration.

Meanwhile, the Vulnerability Percentage (%) chart illustrates the proportion of network traffic identified as malicious or attack traffic by each model. The XGBoost, CatBoost, AdaBoost, and MetaModel showed almost identical results, with approximately 22.5% of the total traffic classified as attacks. The LightGBM model recorded a slightly higher percentage of 26.11%, which may indicate the presence of over-detection or false positives.

Overall, these findings indicate that most models performed consistently in detecting malicious network traffic, achieving high accuracy and stable classification results. The MetaModel (RandomForest), which combines all base models, demonstrated the best performance by effectively integrating their strengths. This result confirms that the ensemble stacking approach is highly effective in improving detection reliability and maintaining a balance between accurate threat identification and minimizing false alarms in network intrusion detection systems.

4.9 Summary

In summary, the meta-model training and evaluation stage ensures that the ensemble not only aggregates multiple predictions but also intelligently learns how to weigh each model’s contribution. Through Random Forest’s ensemble structure and data-driven optimization, the system achieves a balance between interpretability, accuracy, and robustness—making it particularly suitable for complex, high-dimensional datasets such as those used in cybersecurity or network intrusion detection.

4.10 Final Prediction

The Final Prediction stage represents the culmination of the ensemble learning process, where the meta-model produces the ultimate classification output based on the integrated predictions from all base learners. After the training and evaluation phases, the meta-learner—implemented as a Random Forest classifier—serves as the decision-making component that synthesizes the insights derived from *XGBoost*, *CatBoost*, *LightGBM*, and *AdaBoost* base models.

Each base model M_i generates a probabilistic prediction $p_i(x)$ for a given input instance x , where $p_i(x)$ represents the likelihood distribution across possible classes. These outputs are concatenated into a meta-feature vector:

$$z(x) = [p_1(x), p_2(x), p_3(x), p_4(x)]$$

The meta-model function $g(\cdot)$ then processes this feature vector to produce the final output:

$$\hat{y} = g(z(x))$$

where $\hat{y}$ denotes the final class label predicted by the ensemble model. In the context of this research, the function $g(\cdot)$ corresponds to the learned decision function of the Random Forest meta-learner, which aggregates the results of multiple decision trees trained on different bootstrap samples of the meta-feature space.

Each decision tree h_t in the Random Forest contributes an individual prediction $h_t(z(x))$, and the final ensemble decision is derived through a majority voting mechanism:

$$\hat{y} = \text{mode}\{h_1(z(x)), h_2(z(x)), \dots, h_T(z(x))\}$$

where T denotes the total number of trees in the forest. This majority voting ensures that the final output is robust to individual model biases and random variations, resulting in a stable and reliable classification.

From a probabilistic perspective, the final prediction can also be expressed as the expected value over the ensemble of trees:

$$P(\hat{y}|x) = \frac{1}{T} \sum_{t=1}^T P_t(\hat{y}|z(x))$$

where $P_t(\hat{y}|z(x))$ represents the probability assigned by the t^{th} tree to class $\hat{y}$. The predicted label is then obtained as the class with the highest posterior probability:

$$\hat{y} = \arg \max_{c \in C} P(\hat{y} = c|x)$$

where C denotes the set of possible classes. This formulation ensures that the final decision accounts for the collective confidence of the ensemble rather than relying on a single deterministic rule.

The advantage of this approach lies in its ability to balance multiple predictive perspectives. The base models—each trained with different boosting strategies—provide complementary information about the underlying data distribution. The meta-model then learns the optimal combination of these probabilistic outputs, capturing higher-order interactions and correcting individual model weaknesses. Consequently, the final prediction benefits from both the diversity and the collective intelligence of the ensemble.

In practical implementation, the final output from the meta-model can be interpreted as the model’s decision on whether a network flow instance represents normal or malicious activity. For multi-class problems, the same formulation applies, where each class probability is derived from the aggregated vote distribution. The model’s robustness arises from the averaging effect of multiple weak and strong learners, which reduces variance and enhances generalization across unseen data.

The entire prediction pipeline can be summarized as follows:

1. Each base model M_i receives input features X and generates probability outputs $p_i(x)$.
2. The probability vectors from all base models are concatenated into a unified meta-feature $z(x)$.
3. The Random Forest meta-model $g(\cdot)$ processes $z(x)$ through multiple decision trees.
4. Each tree votes on the final class, and the aggregated result determines the predicted label $\hat{y}$.

Mathematically, the final stacked ensemble function can be expressed as:

$$\hat{y}_{final} = g([M_1(x), M_2(x), M_3(x), M_4(x)])$$

This equation encapsulates the hierarchical learning process where the first layer (base learners) captures various feature relationships through boosting algorithms, while the second layer (Random Forest meta-learner) integrates these predictions into a cohesive, accurate decision boundary.

5 Conclusion

This ensemble learning framework integrates four advanced boosting algorithms—XGBoost, CatBoost, LightGBM, and AdaBoost—as base models and Random Forest as the meta-learner, forming a dual ensemble architecture that leverages the strengths of both boosting and bagging. The mathematical formulation demonstrates how base predictions are transformed into meta-features and subsequently refined through averaging across Random Forest trees. This design achieves a powerful balance between bias reduction, variance control, and generalization. The final prediction stage transforms diverse model outputs into a single optimized classification, combining the interpretability of tree-based

learners with the precision of ensemble averaging. This results in a predictive system that is not only accurate and consistent but also resilient to data noise, overfitting, and variability across network traffic patterns.

Overall, the proposed model embodies the principle that an intelligent ensemble of diverse learners often surpasses the performance of any single sophisticated model, delivering robust and high-precision predictions across a wide range of machine learning tasks.

References

- [1] M. L. Audemard, Et al, “Computing abductive explanations for boosted trees,” Arxiv, 2022.
- [2] O. L. Panadero, Et al, “Implementing local-explainability in gradient boosting trees: Feature contribution,” Arxiv, 2024.
- [3] L. F. Chatterjee, Et al, “Tree boosting for learning eft parameters,” Arxiv, 2022.
- [4] G. Chen, “Xgboost: A scalable tree boosting system,” Arxiv, 2016.
- [5] G. V. Prokhorenkova, Et al, “Catboost: unbiased boosting with categorical features,” Arxiv, 2019.
- [6] L. H. Yang, Et al, “Lightgbm robust optimization algorithm based on topological data analysis,” Arxiv, 2024.
- [7] B. Beja, “Overview of adaboost: Reconciling its views to better understand its dynamics,” Arxiv, 2023.
- [8] W. Soloff, Barber, “Bagging provides assumption-free stability,” Arxiv, 2015.
- [9] Biau., “A random forest guided tour,” Arxiv, 2015.
- [10] Breiman, “Bias, variance , and arcing classifiers,” Technical Report 460, Statistics Department, University of California, 2023.
- [11] Chethuhn, “Network intrusion dataset.” <https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset>, 2023. Accessed: 2025-10-15.
- [12] Solarmainframe, “Ids intrusion csv dataset.” <https://www.kaggle.com/datasets/solarmainframe/ids-intrusion-csv>, 2023. Accessed: 2025-10-15.
- [13] E. K. Dhany, “Cse-cic-ids2018 cleaned dataset.” <https://www.kaggle.com/datasets/ekkykharismadhany/csecicids2018-cleaned>, 2023. Accessed: 2025-10-15.
- [14] D. Dhoogla, “Cse-cic-ids2018 dataset.” <https://www.kaggle.com/datasets/dhoogla/csecicids2018>, 2023. Accessed: 2025-10-15.
- [15] D. Dhoogla, “Usb ids version 1 dataset.” <https://www.kaggle.com/datasets/dhoogla/usbids1>, 2023. Accessed: 2025-10-15.